

CSA1718 AI
LAB PROBLEMS

SANJAI.S(192511018)

EXPERIMENT 5:

Vacuum Cleaner PROBLEM

Aim

To implement the Vacuum Cleaner Problem using Python and simulate a simple intelligent agent that cleans all dirty rooms.

Procedure

1. Define the status of Room A and Room B (Clean/Dirty).
2. Set the initial location of the vacuum cleaner.
3. Check the current room.
4. If the room is dirty, clean it.
5. Move to the next room.
6. Repeat until both rooms are clean.
7. Display the final result.

Python Code

A = "Dirty"

B = "Clean"

location = "A"

while True:

 if location == "A":

 if A == "Dirty":

 print("Cleaning Room A")

 A = "Clean"

 print("Moving to Room B")

 location = "B"

 elif location == "B":

 if B == "Dirty":

```
print("Cleaning Room B")
```

```
B = "Clean"
```

```
print("Moving to Room A")
```

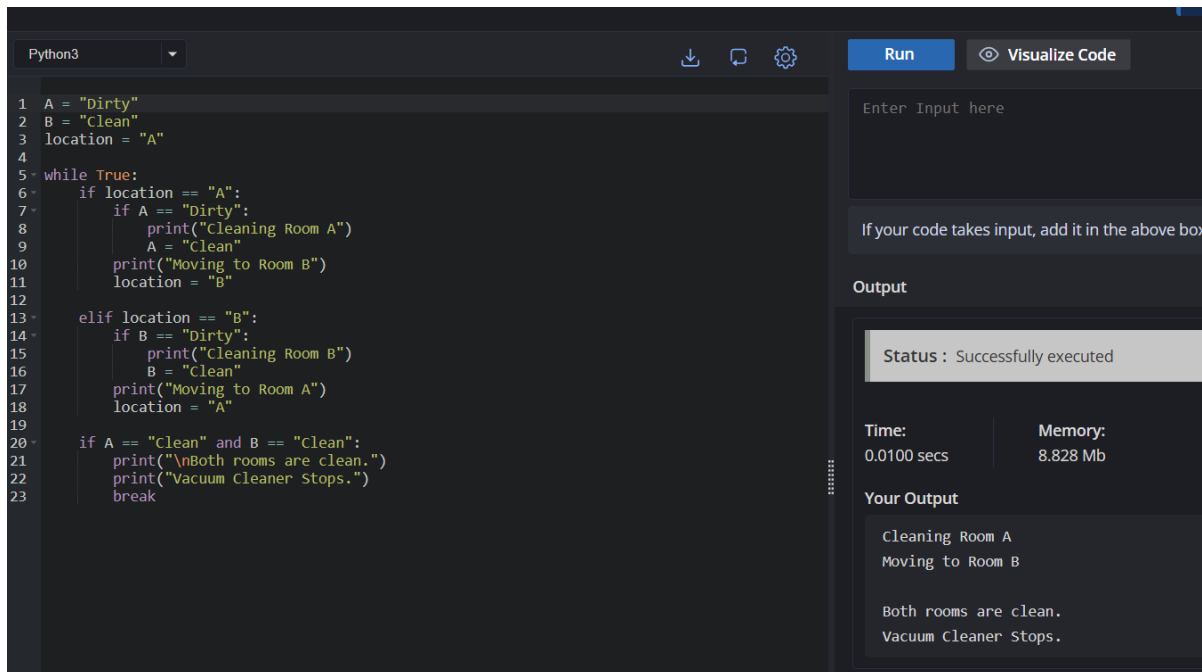
```
location = "A"
```

```
if A == "Clean" and B == "Clean":
```

```
    print("\nBoth rooms are clean.")
```

```
    print("Vacuum Cleaner Stops.")
```

```
    break.
```



The screenshot shows a Python IDE with a dark theme. The left pane contains the following Python code:

```
1 A = "Dirty"
2 B = "Clean"
3 location = "A"
4
5 while True:
6     if location == "A":
7         if A == "Dirty":
8             print("Cleaning Room A")
9             A = "Clean"
10            print("Moving to Room B")
11            location = "B"
12
13        elif location == "B":
14            if B == "Dirty":
15                print("Cleaning Room B")
16                B = "Clean"
17                print("Moving to Room A")
18                location = "A"
19
20    if A == "Clean" and B == "Clean":
21        print("\nBoth rooms are clean.")
22        print("Vacuum Cleaner Stops.")
23        break
```

The right pane shows the execution environment. At the top, there are buttons for "Run" and "Visualize Code". Below them is an input field labeled "Enter Input here" and a note: "If your code takes input, add it in the above box". The "Output" section shows a status message: "Status : Successfully executed". Below this, a table displays execution metrics:

Time:	Memory:
0.0100 secs	8.828 Mb

Under the "Your Output" section, the following text is displayed:

```
Cleaning Room A
Moving to Room B

Both rooms are clean.
Vacuum Cleaner Stops.
```

Result

The program successfully simulates the Vacuum Cleaner Agent. It cleans all dirty rooms and stops after both rooms become clean

Experiment : 6

Missionaries and Cannibals Problem

Aim

To solve the Missionaries and Cannibals problem using the Breadth-First Search (BFS) algorithm in Python.

Procedure

1. Represent the initial and goal states.
2. Define all valid boat movements.
3. Check whether each state is valid.
4. Use Breadth-First Search (BFS) to explore possible states.
5. Avoid revisiting previously explored states.
6. Continue until the goal state is reached.
7. Display the sequence of states from the initial state to the goal state.

Python Code

```
from collections import deque
```

```
def is_valid(m_left, c_left):
```

```
    m_right = 3 - m_left
```

```
    c_right = 3 - c_left
```

```
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
```

```
        return False
```

```
    if m_left > 0 and c_left > m_left:
```

```
        return False
```

```
    if m_right > 0 and c_right > m_right:
```

```
        return False
```

```
    return True
```

```
def bfs():
```

```
    start = (3, 3, 'L')
```

```
goal = (0, 0, 'R')
```

```
queue = deque([(start, [])])
```

```
visited = set()
```

```
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
```

```
while queue:
```

```
    (m, c, boat), path = queue.popleft()
```

```
    if (m, c, boat) == goal:
```

```
        return path + [(m, c, boat)]
```

```
    if (m, c, boat) in visited:
```

```
        continue
```

```
    visited.add((m, c, boat))
```

```
    for dm, dc in moves:
```

```
        if boat == 'L':
```

```
            new = (m-dm, c-dc, 'R')
```

```
        else:
```

```
            new = (m+dm, c+dc, 'L')
```

```
        if is_valid(new[0], new[1]):
```

```
            queue.append((new, path + [(m, c, boat)]))
```

```
solution = bfs()
```

```
print("Missionaries and Cannibals Solution:\n")
```

```
for step in solution:
```

```
    print(step)
```

```

from collections import deque

def is_valid(m_left, c_left):
    m_right = 3 - m_left
    c_right = 3 - c_left

    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False

    if (m_left > 0 and c_left > m_left):
        return False

    if (m_right > 0 and c_right > m_right):
        return False

    return True

def bfs():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')

    queue = deque([(start, [])])
    visited = set()

    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

    while queue:
        (m, c, boat), path = queue.popleft()

        if (m, c, boat) == goal:
            return path + [(m, c, boat)]

        if (m, c, boat) in visited:
            continue

        visited.add((m, c, boat))

        for dm, dc in moves:
            if boat == 'L':
                new = (m-dm, c-dc, 'R')
            else:
                new = (m+dm, c+dc, 'L')

            if is_valid(new[0], new[1]):
                queue.append((new, path + [(m, c, boat)]))

```

Enter Input here

If your code takes input, add it in the input box above

Output

Status : Successfully executed

Time:

0.0100 secs

Memory:

9.888 Mb

Your Output

Missionaries and Cannibals

```

(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 0, 'R')

```

Result

The program successfully finds a valid sequence of moves to transport all three missionaries and three cannibals across the river without violating the problem constraints using the Breadth-First Search (BFS) algorithm.